Operating Systems: File Systems Architecture & Disk I/O Management

Inodes, Directory Structures, Virtual File System (VFS), Journaling, and Disk Scheduling

Doc Id: OS-FS-2024-004 | **Practice:** File Systems & Block I/O Subsystem | **System:** Linux VFS, ext4, ZFS, NVMe I/O Subsystem | **Year:** 2024

1. Inode Architecture and File Allocation Schemes

In UNIX-derived file systems, files are represented by an **Index Node (Inode)**. The inode stores all file metadata (file size, permissions, owner UID/GID, timestamps atime/mtime/ctime, reference link count) and block address pointers, separating metadata from filename directory mappings.

• **Direct and Indirect Blocks:** Traditional UFS/ext2 inodes used 12 direct block pointers, 1 single-indirect pointer, 1 double-indirect pointer, and 1 triple-indirect pointer.

• **Extents (ext4):** Replaces individual block pointers with extents (start_block, block_count). A single extent can address up to 128 MB of contiguous disk blocks, reducing metadata fragmentation and tree depth.

• **Hard Links vs. Soft (Symbolic) Links:** A hard link is another directory entry pointing directly to the existing inode number (increments link count; file data persists until link count is 0). A soft link is an independent file whose data payload contains the text path to the target file.

Feature	Contiguous Allocation	Linked List Allocation	Indexed (Inode) Allocation	Extent-Based (ext4 / XFS)
Sequential Read Speed	Maximum (contiguous sectors)	Moderate (follows block pointers)	High (block pointers in memory)	Ultra-High (contiguous multi-megabyte)
Random Access Speed	O(1) direct block offset	O(N) traverse block links	O(1) through pointer tree	O(1) through extent B-tree
External Fragmentation	Severe (requires compaction)	None	None	Low (addressed by preallocation)
File Size Growth	Difficult (must move or allocate max)	Simple (append new block)	Simple (allocate indirect blocks)	Efficient (grow extent)
Reliability / Corruption	High	Critical (broken pointer loses file)	Robust (inode integrity checks)	High (checksummed extent trees)

2. Virtual File System (VFS) Layer

The Virtual File System (VFS) provides an object-oriented kernel abstraction layer allowing standard POSIX syscalls (open, read, write, close) to operate seamlessly across heterogeneous file systems (ext4, XFS, NFS, procfs, sysfs).

The VFS defines four core primary data structures:

1. **Superblock Object:** Represents an entire mounted file system (block size, total blocks, magic number, root dentry).

2. **Inode Object:** Represents a specific file or directory metadata entity.

3. **Dentry (Directory Entry) Object:** Represents a directory entry linking a pathname string component to its inode. Managed in an in-memory **dentry cache (dcache)** for rapid path lookups.

4. **File Object:** Represents an open file instance created upon open(), storing the current file offset pointer (f_pos), access modes, and pointer to the dentry.

3. File System Journaling and Crash Consistency

A crash occurring during a multi-block file write (e.g. data block written, but inode size not yet updated) causes file system corruption.

Journaling file systems maintain a dedicated circular write-ahead log (journal) on disk to ensure atomic metadata transitions:

• **Journal Mode:** Both metadata and file payload data are written to the journal before being committed to the main file system. Highest integrity, lowest throughput.

• **Ordered Mode (ext4 default):** File payload data is flushed to disk first; once complete, metadata updates are logged to the journal. Prevents garbage data exposure on crash with near-metadata speed.

• **Writeback Mode:** Only metadata is journaled; data order is unconstrained. High speed, but crash can leave newly allocated blocks with old data.

Journaling Mode	Crash Recovery Guarantee	Write Amplification	Throughput Relative to Raw Disk	Default System
Full Journal	Zero data loss; atomic data + metadata	2.0x (every byte written twice)	45 - 55%	Specialized audit / safety logs
Ordered Mode	Metadata consistency; no stale data leaks	1.0x data + small metadata journal	88 - 94%	Linux ext4, ext3 default
Writeback Mode	Metadata consistency only; data may corrupt	1.0x data + minimal metadata journal	95 - 99%	High-performance scratch disks
Copy-on-Write (ZFS / Btrfs)	Always consistent; no in-place overwrite	Variable (tree commit updates)	85 - 92%	Enterprise storage pools / snapshots

4. Disk Scheduling Algorithms: Rotational and SSD I/O

Disk scheduling reorders pending block I/O requests to minimize physical seek time and rotational latency:

• **SSTF (Shortest Seek Time First):** Selects request closest to current head position. Minimizes average response time, but risks starving distant tracks.

• **SCAN (Elevator Algorithm):** Arm moves in one direction servicing all requests until it reaches the outer end, then reverses direction.

• **C-SCAN (Circular SCAN):** Moves in one direction servicing requests; upon reaching the end, immediately returns to the start without servicing on return, providing uniform wait time distribution.

• **NVMe / SSD Schedulers:** Solid-State Drives have zero mechanical seek time. Schedulers like Linux none / mq-deadline bypass rotational reordering in favor of multi-queue parallel hardware completion rings (up to 64K queues with 64K depth each).